

Introdução à Arquitetura de Microserviços

Programação Distribuída / José Marinho – 2025/2026

O que são Microserviços?

- Definição
 - Abordagem arquitetural onde a aplicação é uma coleção de pequenos serviços independentes/autónomos que trabalham juntos
 - Cada serviço foca-se numa funcionalidade de negócio específica (e.g., carrinho de compras, autenticação)
- Funcionamento
 - Correm de forma autónoma (processos independentes).
 - Comunicam através de mecanismos leves (API HTTP/REST, mensagens)

2

Programação Distribuída / José Marinho – 2025/2026

Contexto Histórico

- Origem
 - Surgiu como resposta às limitações das arquiteturas monolíticas e SOA (*Service-Oriented Architecture*) dos anos 2000
 - Popularizado em 2014 (artigo de Martin Fowler) e adotado por empresas como Netflix e Amazon para resolver problemas de escala
- Evolução
 - Sistema monolítico: bloco único (ex.: tudo num só contentor)
 - Microsserviços: unidades funcionais com implantação/colocação em produção independente (ex.: em contentores Docker)

3

Programação Distribuída / José Marinho – 2025/2026

Arquitetura Monolítica vs. Microsserviços

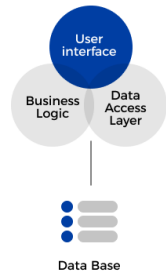
- Arquitetura monolítica
 - Base de código única e fortemente acoplada
 - Base de dados partilhada por todos os módulos
 - Problema:
 - Uma falha num módulo pode comprometer todo o sistema
 - A colocação em produção exige atualizar tudo de uma vez
- Arquitetura de microsserviços
 - Fracamente acoplados (*Loosely coupled*)
 - Base de dados por serviço (descentralizada)
 - Vantagem: falhas isoladas e implantação independente

4

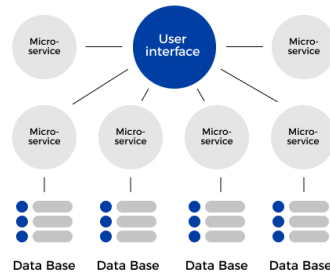
Programação Distribuída / José Marinho – 2025/2026

Arquitetura Monolítica vs. Microserviços

MONOLITHIC ARCHITECTURE



MICROSERVICE ARCHITECTURE



<https://www.cloudflight.io/en/blog/monolithic-architecture-vs-microservices/>

5

Programação Distribuída / José Marinho – 2025/2026

Características Principais

- Independência
 - Cada serviço pode ser desenvolvido, atualizado e escalado sem afetar os outros
 - Permite “independência tecnológica” (ex.: um serviço em Java e outro em Python)
- Granularidade e domínio
 - Foco num domínio de negócio específico (ex: serviço de encomendas)
 - Abordagem *Domain-Driven Design* (DDD)

6

Programação Distribuída / José Marinho – 2025/2026

Características Principais

- Gestão descentralizada de dados
 - Cada microsserviço gere a sua própria base de dados
 - Evita o acoplamento excessivo gerado por bases de dados partilhadas
- Produtos, não projetos
 - A equipa é responsável por todo o ciclo de vida do serviço: construção, implantação e manutenção em produção
 - Fomenta a cultura DevOps (“You build it, you run it”: responsabilidade total desde o código até à produção)

7

Programação Distribuída / José Marinho – 2025/2026

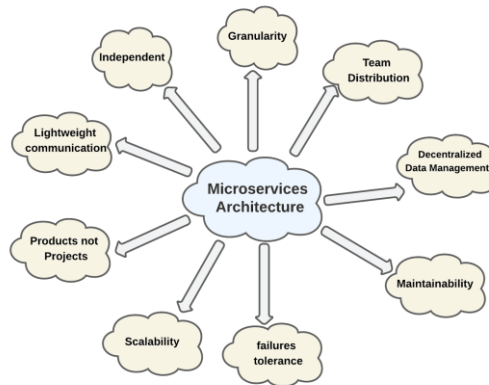
Benefícios Principais

- Escalabilidade melhorada
 - Escala-se apenas o serviço que tem procura elevada, não a aplicação toda (poupança de recursos)
- Isolamento de falhas (resiliência)
 - Se, por exemplo, o serviço de “carrinho de compras” falhar, o utilizador ainda pode navegar no “catálogo”
- Agilidade no desenvolvimento
 - Equipas pequenas e focadas trabalham em paralelo
 - *Time-to-market* mais rápido

8

Programação Distribuída / José Marinho – 2025/2026

Benefícios Principais



<https://dev.to/josueparra2892/chapter-1-introduction-to-microservices-5a2c>

9

Programação Distribuída / José Marinho – 2025/2026

Desafios

- Complexidade inerente aos sistemas distribuído
 - A comunicação entre serviços introduz latência e possibilidade de falhas de rede
- Consistência de dados
 - Manter dados sincronizados entre bases de dados separadas pode ser complexo (transações distribuídas)
- Monitorização e *debugging*
 - É complexo rastrear um erro que envolve o recurso a vários serviços
 - Exige técnicas de *distributed tracing* e *logs* centralizados

10

Programação Distribuída / José Marinho – 2025/2026

Escalabilidade: Independência e Granularidade

- Ao contrário da arquitetura monolítica, onde escalar significa replicar toda a aplicação
 - Os microsserviços permitem escalabilidade granular
 - Cada componente pode ser escalado individualmente com base na sua procura específica
- Eficiência de recursos
 - Permite alocar mais recursos apenas onde é necessário (ex.: escalar apenas o serviço de “pagamentos” na Black Friday)
 - Evita o desperdício de infraestrutura e custos associados a escalar módulos pouco solicitados ou geralmente inativos

11

Programação Distribuída / José Marinho – 2025/2026

Escalabilidade: Independência e Granularidade

- Elasticidade
 - O sistema adapta-se rapidamente a cargas de trabalho variáveis, aumentando ou diminuindo instâncias conforme a necessidade

12

Programação Distribuída / José Marinho – 2025/2026

Escalabilidade: Mecanismos Técnicos

- Escalabilidade horizontal
 - Criação de múltiplas instâncias do mesmo serviço distribuídas por diferentes servidores ou contentores
 - Diferente da escalabilidade vertical (aumentar o *hardware* de uma única máquina)
 - Esta abordagem é praticamente ilimitada
- Serviços *stateless*
 - Não guardam informações do cliente localmente entre pedidos
 - Facilita a replicação: qualquer instância do serviço pode tratar qualquer pedido

13

Programação Distribuída / José Marinho – 2025/2026

Escalabilidade: Mecanismos Técnicos

- Equilíbrio de carga
 - Distribuição do tráfego de rede de entrada uniformemente entre as várias instâncias operacionais do serviço
 - Previne que uma única instância fique sobrecarregada
 - Contribuindo também para a tolerância a falhas
- Orquestração de contentores
 - Gestão automatizada do ciclo de vida de contentores (implementação, gestão, escalabilidade e rede)
 - Ferramentas como Kubernetes (K8s) automatizam este processo com base em diversas métricas (ex.: CPU ou tráfego)

14

Programação Distribuída / José Marinho – 2025/2026

Escalabilidade: Organizacional

- Para além do *hardware*, a arquitetura de microsserviços também resolve problemas de escala humana e de gestão de equipas
- Equipas autónomas
 - Permite distribuir equipas de desenvolvimento por diferentes serviços (domínios)
 - As equipas podem crescer e trabalhar em paralelo sem criar conflitos de *codebase* ou bloqueios de colocação em produção, comuns em equipas grandes a trabalhar numa única solução monolítica

15

Programação Distribuída / José Marinho – 2025/2026

Escalabilidade: Organizacional

- Adoção de tecnologias heterogéneas
 - Permite escalar a tecnologia de forma heterogénea
 - Por exemplo, um serviço que precisa de desempenho matemático elevado pode ser escrito em C++, enquanto outro focado em *Input/Output* pode usar Node.js

16

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Fundamentos

- Os microsserviços comunicam geralmente via HTTP ou mensagens, garantindo independência tecnológica
- Contratos de API
 - Cada serviço expõe uma API bem definida
 - Definem *endpoints*, métodos, formatos de dados (ex.: JSON) e mecanismos de autenticação
- Dois tipos principais de comunicação
 - Síncrono: o cliente espera pela resposta (ex.: REST e gRPC)
 - Assíncrono: o cliente envia o pedido e não bloqueia à espera da resposta (ex.: *message queues* e *event-driven*)

17

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Exemplos de Soluções Síncronas

- REST API (mais comum)
 - Baseado em métodos HTTP e recursos (URI)
 - Vantagens: simples, *stateless*, escalável e amplamente suportado
- gRPC (desempenho elevado)
 - *Framework* RPC desenvolvida pela Google
 - Usa *Protocol Buffers* (binário) em vez de JSON ou XML para efeitos de serialização
 - Vantagens: bidirecional, eficiente e suporte de *streaming*
 - Ideal para comunicação entre microsserviços
 - Suporta várias linguagens de programação (ambiente “poliglota”)

18

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Exemplos de Soluções Síncronas

- GraphQL
 - Linguagem de consulta (*query language*) para API
 - Permite que o cliente especifique exatamente a estrutura da resposta
 - Desta forma, o cliente pede apenas os dados de que necessita
 - Evita os efeitos de *over-fetching* (receber dados a mais) ou de *under-fetching* (ter de fazer múltiplos pedidos para obter toda a informação necessária), comuns em abordagens REST
 - Torna a comunicação mais eficiente para o *frontend* (cliente)

19

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Exemplos de Soluções Assíncronas

- Uso de *brokers* como RabbitMQ ou Apache Kafka
 - *Broker*: componente de software que atua como intermediário na comunicação entre microsserviços
- Desacoplamento do serviço produtor do serviço consumidor
- Comunicação assíncrona e fiável
 - O produtor não aguarda pela entrega da mensagem, nem pela receção da resposta: apenas a publica no *broker*
 - O consumidor processa a mensagem/evento quando está livre
 - Se o serviço de destino estiver inacessível ou sobrecarregado, o *broker* guarda as mensagens na fila até que recupere

20

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Exemplos de Soluções Assíncronas

- RabbitMQ
 - *Message broker open-source* popular e leve
 - Funciona como um gestor de tráfego (filas de mensagens) que aceita, armazena e encaminha mensagens entre serviços
 - Desacoplamento temporal: o produtor da mensagem não precisa de saber se o consumidor está *online* no momento do envio
 - Garante que as mensagens são entregues de forma fiável, mesmo que ocorram falhas de rede
 - Utiliza o protocolo AMQP (*Advanced Message Queuing Protocol*) para normalizar a troca de mensagens

21

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Exemplos de Soluções Assíncronas

- Ideal para tarefas que não precisam de resposta imediata (ex.: redimensionamento de uma foto carregada no sistema por um microserviço diferente daquele que fez o *upload*)
- Ajuda a gerir picos de tráfego, acumulando os pedidos na fila e permitindo que os consumidores os processem ao seu ritmo
- Uma única mensagem pode ser enviada para o RabbitMQ e distribuída para múltiplos serviços interessados simultaneamente (ex.: uma nova compra pode desencadear o aviso dos microserviços de “stock” e de “envio” ao mesmo tempo)

22

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Exemplos de Soluções Assíncronas

- Apache Kafka
 - Uma plataforma de *streaming* de eventos distribuída
 - Ao contrário de uma fila simples que "apaga" a mensagem depois de lida (ex.: como no RabbitMQ), o Kafka funciona como um registo (*log*) durável de eventos: armazena o histórico dos eventos por um período definido
 - Múltiplos serviços podem ler o mesmo evento de forma independente e em momentos diferentes
 - *Event sourcing*: como o Kafka guarda o histórico de eventos, um novo microsserviço pode observar o que aconteceu antes de correr/existir

23

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Exemplos de Soluções Assíncronas

- Ideal para processar volumes elevados de dados em tempo real (ex.: *logs*, telemetria de sensores IoT) onde a velocidade é crítica
- Modelo *Publisher / Subscriber*
 - Arquitetura orientada a eventos (*event-driven*)
 - O microsserviço produtor publica eventos em determinados tópicos
 - Outros microsserviços (consumidores) subscrevem esses tópicos e leem os eventos à medida que ocorrem
 - Exemplo: o serviço de “encomendas” publica um evento “encomenda criada” e o serviço de “envios” reage a este, sem que se “conheçam” diretamente

24

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Exemplos de Soluções Assíncronas

- Uma ação (publicação de um evento) pode desencadear reações em múltiplos serviços desconhecidos pelo produtor
 - Exemplo: quando um utilizador publica o evento “cria conta”, o Kafka distribui esse evento e o serviço de “email” manda as boas-vindas, o serviço de “analytics” regista o cadastro e o serviço de “customer relationship management” cria o perfil, ao mesmo tempo

25

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Exemplos de Soluções Assíncronas

- MQTT (*Message Queuing Telemetry Transport*)
 - Protocolo de troca de mensagens leve, apropriado para dispositivos com poucos recursos (bateria, CPU) e redes instáveis ou com largura de banda limitada
 - Usado para ligar dispositivos remotos (ex.: sensores) aos microserviços ($\neq$ comunicação inter-serviços)
 - Ambientes de *Edge Computing* e IoT (*Internet of Things*)
 - Utiliza também o padrão *Publisher / Subscriber*, permitindo uma comunicação desacoplada e eficiente

26

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Exemplos de Soluções Assíncronas

- Numa arquitetura de microsserviços, o *broker* MQTT atua como porta de entrada (*ingress*) dos dados
- Fluxo típico:
 - Dispositivos: enviam dados via MQTT
 - Broker MQTT: recebe as mensagens
 - Microsserviço "ponte": consome as mensagens do MQTT e coloca-as, por exemplo, no Kafka ou numa base de dados para processamento posterior

27

Programação Distribuída / José Marinho – 2025/2026

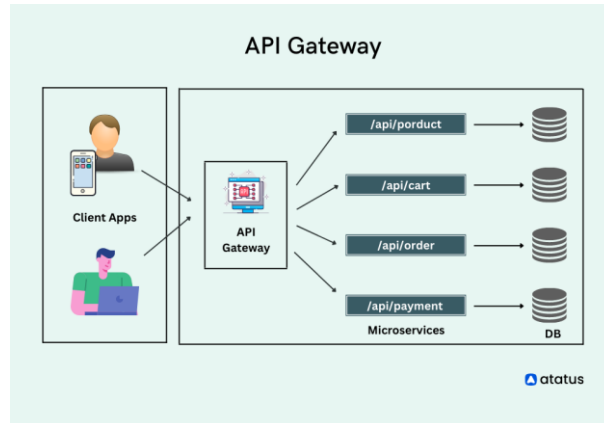
Comunicação: Padrões de Arquitetura

- *API Gateway*
 - Ponto de entrada único para clientes externos (web/mobile)
 - Isola o cliente da complexidade interna da rede de microsserviços
 - Encaminha pedidos, trata da autenticação e distribuição de carga, traduz protocolos e agrega respostas de vários serviços (*API composition*)
- *Service Registry & Discovery*
 - Funciona como uma "lista telefónica" dinâmica
 - Os serviços registam-se ao arrancar e consultam o registo para obter os endereços de rede (IP e porto) uns dos outros

28

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Padrões de Arquitetura

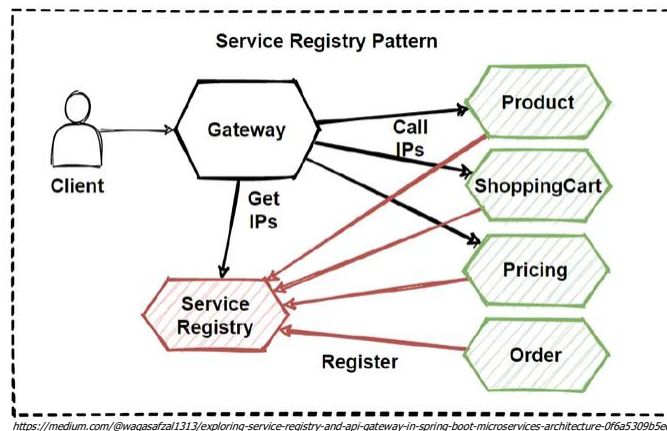


<https://peerdh.com/blogs/programming-insights/api-gateway-patterns-in-microservices-for-e-commerce>

29

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Padrões de Arquitetura



<https://medium.com/@waqasafzal1313/exploring-service-registry-and-api-gateway-in-spring-boot-microservices-architecture-0f6a5309b5e0>

30

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Desafios e Resiliência

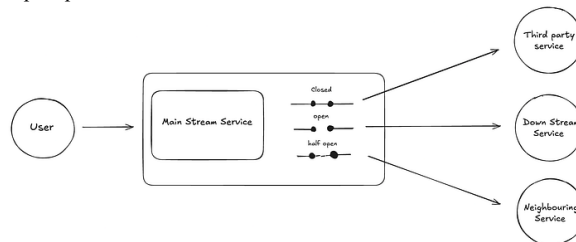
- Falhas de rede
 - Num sistema distribuído, a rede não é fiável
 - Implementar *timeouts* e *retries* automáticos é essencial
- *Circuit breaker* (disjuntor) *pattern*
 - Impede que uma falha num serviço se propague a todo o sistema
 - Se um serviço falhar n vezes, o circuito "abre" e para de encaminhar pedidos temporariamente
 - Monitoriza os serviços em estado de falha e reinicia o fluxo quando a conexão é recuperada
 - Permite ao sistema recuperar graciosamente

31

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Desafios e Resiliência

- Estados do *circuit breaker*
 - Fechado: todas as chamadas no sistema de disjuntor são permitidas
 - Aberto: o disjuntor impede todas as chamadas no sistema
 - Semiaberto: é permitida uma pequena parte das chamadas até que se verifique que o sistema volte a funcionar corretamente

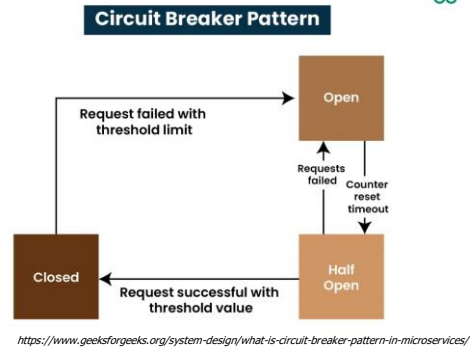


<https://medium.com/sisalhubs/what-is-the-circuit-breaker-pattern-and-why-is-it-important-for-microservice-architecture-6c997be77949>

32

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Desafios e Resiliência

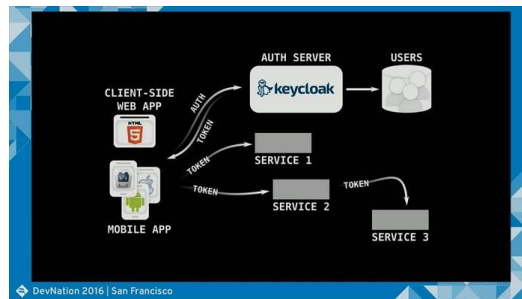


33

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Desafios e Resiliência

- Uso de *tokens* de acesso (ex.: OAuth 2.0, JWT) para gerir autenticação entre serviços

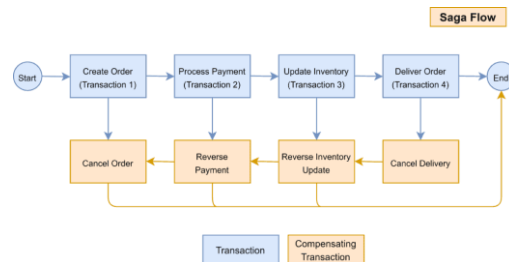


34

Programação Distribuída / José Marinho – 2025/2026

Comunicação: Desafios e Resiliência

- *Saga Pattern*
 - Solução para transações distribuídas (vários passos locais)
 - Se um passo falhar, executa ações de compensação para desfazer o anterior



<https://www.baeldung.com/cs/saga-pattern-microservices>

35

Programação Distribuída / José Marinho – 2025/2026

Recomendações/Boas Práticas

- *Domain-Driven Design*
 - Estruturar os microsserviços em torno de domínios de negócio específicos
 - Definir fronteiras claras para evitar serviços demasiado grandes ou demasiado fragmentados
- *Database per Service*
 - Cada microsserviço deve possuir e gerir a sua própria base de dados para garantir o desacoplamento total

36

Programação Distribuída / José Marinho – 2025/2026

Recomendações/Boas Práticas

- Comunicação e contratos
 - Utilizar API REST ou soluções de comunicação assíncrona para garantir comunicação leve e independente da linguagem
 - Garantir a retrocompatibilidade (*backward compatibility*) das API para que atualizações num serviço não quebrem o acesso dos consumidores
- Configuração Dinâmica
 - Evitar valores *hardcoded* (ex.: endereços IP)
 - Utilizar mecanismos de *Service Discovery* para que os serviços se localizem dinamicamente na rede

37

Programação Distribuída / José Marinho – 2025/2026

Recomendações/Boas Práticas

- Automação (CI/CD)
 - Implementar pipelines de integração e entrega contínuas para automatizar testes e colocações em produção
 - Assegurar que cada serviço pode ser implantado de forma independente, sem coordenação complexa com outros serviços
- Observabilidade
 - Investir em ferramentas de monitorização que centralizem métricas (ex.: Prometheus, Grafana), *logs* (ex.: ELK Stack - Elasticsearch) e rastreio distribuído (ex.: Jaeger)
 - Essencial para identificar a causa de problemas num ambiente distribuído complexo

38

Programação Distribuída / José Marinho – 2025/2026

Recomendações/Boas Práticas

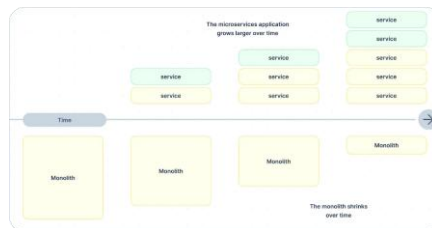
- Segurança
 - Implementar autenticação e autorização (ex.: OAuth 2.0) em cada interação de serviço
 - Encriptar dados em trânsito
- Cultura e autonomia
 - Adotar uma estrutura organizacional que promova a autonomia das equipas e a responsabilidade total pelo ciclo de vida do serviço ("You build it, you run it")

39

Programação Distribuída / José Marinho – 2025/2026

Recomendações/Boas Práticas

- Estratégia de migração baseada no *Strangler Pattern*
 - Substituir gradualmente funcionalidades do sistema monólito por novos microserviços
 - O sistema monólito "encolhe" com o tempo até desaparecer ou ficar residual



<https://www.multiplexer.app/distributed-systems-architecture/a-guide-to-microservices-design-pattern/>

40

Programação Distribuída / José Marinho – 2025/2026

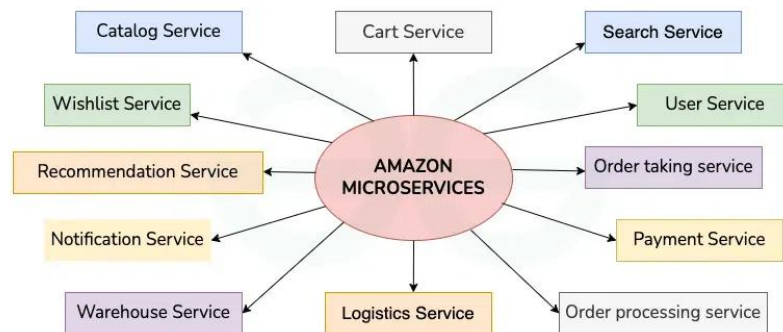
Exemplo: Amazon (E-Commerce)

- Antes (2001): abordagem monolítica que dificultava a escalabilidade
- Atualmente: centenas de microsserviços especializados
 - *User Service*: contas e preferências
 - *Catalog Service*: listagem de produto
 - *Order Service*: processamento de encomendas
 - *Recommendation Service*: sugestões personalizadas
 - etc.

41

Programação Distribuída / José Marinho – 2025/2026

Exemplo: Amazon (E-Commerce)



Amazon e-commerce Microservices

<https://www.geeksforgeeks.org/system-design/microservices/>

42

Programação Distribuída / José Marinho – 2025/2026

Exemplo: Netflix (*Streaming*)

- Limitações (2008)
 - Corrupção da base de dados e interrupções de serviço devido à arquitetura monolítica e *data centers* tradicionais
 - Dificuldade em escalar para milhões de utilizadores
- Solução: microsserviços + AWS (*Amazon Web Services*)
 - Migração completa para a *cloud* (AWS) e adoção de microsserviços iniciada em 2009
 - AWS: permite que as equipas se foquem no código do serviço e não na instalação e manutenção da infraestrutura (ex.: Kafka)
 - Atualmente, opera com mais de 500 microsserviços

43

Programação Distribuída / José Marinho – 2025/2026

Exemplo: Netflix (*Streaming*)

- Exemplos de serviços:
 - *Video Playback Service*: gere a reprodução do conteúdo
 - *Recommendation Service*: algoritmos para sugerir filmes e séries.
 - *Search Service*: pesquisa de conteúdo
 - *User Profile Service*: gestão de perfis e preferências

44

Programação Distribuída / José Marinho – 2025/2026

Exemplo: Uber (Mobilidade)

- A aplicação divide-se em serviços críticos que operam em tempo real
- Exemplos de serviços:
 - *Passenger Management*: gere dados e histórico dos utilizadores
 - *Driver Management*: perfis, estatísticas e estado dos motoristas
 - *Geolocation Service*: rastreio em tempo real e mapas
 - *Trip Assignment*: algoritmo de *matching* entre condutor e passageiro
 - *Payment Processing*: gestão de transações e pagamentos

45

Programação Distribuída / José Marinho – 2025/2026

Exemplo: Twitter/X (Rede Social)

- Desafio: gerir milhões de pedidos e *tweets* em tempo real com latência mínima
- Arquitetura: decomposição em serviços para recuperação rápida de dados
- Exemplos de serviços:
 - *Timeline Service*: constrói o *feed* do utilizador
 - *Mentions & Notifications*: processamento assíncrono de interações
 - *Trends Service*: análise de tópicos populares em tempo real
 - *Search Service*: indexação e pesquisa de tweets

46

Programação Distribuída / José Marinho – 2025/2026

Exemplo: Airbnb (Alojamento local)

- Contexto: plataforma global de aluguer *peer-to-peer* que necessita de alta disponibilidade
- Exemplos de Serviços:
 - *Search & Booking*: o núcleo do negócio, gerido independentemente
 - *Payment Management*: processamento complexo de pagamentos em múltiplas moedas
 - *Review & Rating*: Serviço dedicado apenas às avaliações e classificações
- Facilidade de introdução de novas funcionalidades

47

Programação Distribuída / José Marinho – 2025/2026

Conclusão

- Os microsserviços oferecem agilidade, escalabilidade e resiliência, mas exigem maturidade operacional
- Adequados para sistemas complexos, em rápida evolução e com necessidade de escalar
- Para aplicações simples, a abordagem monolítica ainda é válida e mais fácil de gerir

48

Programação Distribuída / José Marinho – 2025/2026